Security Assessment Report

Generated: 2026-05-05 11:11
Report ID: bd0338be

Security Assessment Report

Target: /workspace/uploads/ebc94e7b2c95

Assessment Type: Automated Static Security Analysis

Tools Used: Bandit, SonarQubestyle analysis, Safety

Assessment Date: 20260505

1. Executive Summary

The automated security assessment of the provided project directory identified no confirmed vulnerabilities from the available static analysis results. Bandit reported zero security findings across the analyzed Python source file, and the Sonarstyle scan returned no issues.

However, the assessment is limited by the available evidence:

- The Safety scan could not execute meaningfully because no requirements.txt file was present.
- No dependency manifest was available for package vulnerability review.
- The scan results only confirm that no obvious issues were detected in the inspected code paths, not that the application is free of security risk.

Overall system risk level: Low, based on the current automated results, with the important caveat that incomplete dependency visibility prevents a full endtoend assurance.

2. Risk Overview

Findings by Severity

- Critical: 0
- High: 0
- Medium: 0
- Low: 0

Informational: 1

Risk Interpretation

Critical / High / Medium / Low: No confirmed codelevel vulnerabilities were identified in the automated scans.

Informational: One informational observation was recorded regarding the absence of a Python dependency manifest, which limits software composition analysis and may reduce visibility into thirdparty package risk.

3. Detailed Findings

Informational Findings

Finding 1 — Missing Python Dependency Manifest

Severity: Informational

CVSS Score (estimated): 0.0

OWASP Category:

A06: Vulnerable and Outdated Components

Affected Component:

Project root: /workspace/uploads/ebc94e7b2c95

Dependency scanning input: requirements.txt not present

Description:

The Safety scan reported that no requirements.txt file was found in the project.

This is not a vulnerability in itself, but it prevents automated validation of Python dependencies for known security issues.

In practice, the absence of a dependency manifest reduces visibility into thirdparty package exposure and may allow vulnerable libraries to go unnoticed.

Impact:

An attacker could potentially exploit vulnerabilities in unmanaged or unreviewed dependencies if they exist in the runtime environment.

From a business perspective, this creates uncertainty in the software supply chain and weakens the organization’s ability to demonstrate dependency hygiene and patch discipline.

Proof of Concept (if possible):

Not applicable. This is an assessment limitation rather than an exploitable weakness.

Recommended Remediation:

Add and maintain a dependency manifest such as requirements.txt, Pipfile, or pyproject.toml.

Pin dependency versions and review them regularly.

Integrate dependency scanning into CI/CD using tools such as Safety, pipaudit, Dependabot, or equivalent.

Establish a process for timely patching of thirdparty libraries and base images.

If dependencies are installed via another mechanism, document the process and ensure they are still scanned.

4. Recommendations (Global)

1. Introduce dependency inventory and scanning

Maintain a clear dependency manifest.

Scan dependencies on each build and on a scheduled basis.

Track transitive dependencies, not only direct packages.

2. Preserve secure coding practices

Continue using input validation, output encoding, least privilege, and strong error handling.

Avoid unsafe dynamic execution patterns and shell invocation where possible.

Ensure sensitive data is never logged in plaintext.

3. Strengthen secure SDLC controls

Add SAST, dependency scanning, and secret scanning to CI/CD.

Review highrisk code changes manually before release.

Require code review for securitysensitive components.

4. Harden runtime security

Run services with minimal privileges.

Use current supported versions of the Python runtime and frameworks.

Apply OS, container, and library updates regularly.

5. Improve security visibility

Generate and retain software bills of materials where practical.

Define patch SLAs for critical and highrisk findings.

Periodically rerun scans after dependency or code changes.

5. Conclusion

The automated assessment did not identify any confirmed codelevel vulnerabilities in the provided project. Based on the results available, the application currently presents a Low security risk profile.

That said, the inability to perform dependency vulnerability analysis due to the absence of a requirements.txt file limits assurance. While there are no immediate critical issues indicated by the scans, it remains important to establish proper dependency management and continue routine security testing to prevent future exposure.

Priority action: implement and maintain a dependency manifest so that thirdparty package risk can be assessed and remediated promptly.